

Assignment 2 — CSS: Selectors, Box Model, Flexbox & Grid

Assignment 2 — CSS: Selectors, Box Model, Flexbox & Grid

Based on: IMAP Lecture 2 (CSS — Cascading Style Sheets)

What you are given

You have been provided two starter files:

- `starter.html` — the same portfolio page from Assignment 1, with a few extra elements already added (a navbar, a sidebar, and project cards) so you have enough to style. It currently has **no visual styling at all** — it will look like plain, unstyled HTML when you open it.
- `starter.css` — an empty stylesheet, organised into commented sections, one per task.

Open `starter.html` in your browser before you begin, so you see the unstyled page.

What you need to do

There are **10 tasks** below. Each task asks you to modify only `starter.css` — **do not touch** `starter.html` for this assignment. Work through them **in order**, on the same file — each task builds on the page as it looks after the previous task. Write your rules in the matching section of the provided `starter.css`.

For **every task**, you must submit:

1. A "**Before**" screenshot — the relevant part of the page as it looked before your change.
2. An "**After**" screenshot — the same part of the page after your change.
3. The **block of CSS you added** for that task, pasted in as code.

You do not need to screenshot the entire page every time — just the section that changed is fine, as long as the change is clearly visible.

Submission format

Submit a single **PDF** named `<Roll No.>_IMAP_Lab2.pdf` (e.g., `J012_IMAP_Lab2.pdf`) containing, for each of the 10 tasks:

```
### Q<number> – <short title>
```

```
Before: [screenshot]
```

```
After:  [screenshot]
```

Code added:

[paste the CSS block you added]

Along with your final `starter.css` file (with all 10 changes applied). `starter.html` must stay unchanged.

Tasks

Q1. Reset & custom properties

At the top of the file, add the universal `box-sizing: border-box` reset (`*`, `*::before`, `*::after`). Then define at least **four custom properties** on `:root` — for example a brand colour, a darker shade of it, a spacing value, and a border-radius value. Use `var()` to apply at least two of them somewhere else in the file (they don't have to be used yet in this task — you will use them again in later tasks).

Q2. Selectors, combinators & pseudo-classes

Using a **class selector**, style the links inside `.nav-links` (colour, remove underline) and add a `:hover` state. Using `:nth-child()`, give every other `.card` in `.cards` a slightly different background colour (a zebra-style effect). Add a visible `:focus-visible` outline style to the form's `input` and `textarea` elements — never remove the default outline without replacing it.

Q3. Box model

Give `.block` and `.card` some breathing room: add `padding`, a `border` (or `box-shadow`), and a `border-radius` so each one reads as a clear, separated box. Adjust `margin` where needed so sections and cards don't feel cramped against each other.

Q4. Typography

Set a `font-family` stack on `body` (end it with a generic family like `sans-serif`). Give headings and body text distinct, deliberate `font-size` and `font-weight` values using `rem`, and set a comfortable `line-height` (e.g. `1.5` for body text). Constrain paragraph text inside `.block` to a readable width with `max-width: 65ch`.

Q5. Flexbox: navbar

Turn `.navbar` into a flex container. Place `.brand` on the left and `.nav-links` on the right using `justify-content: space-between`, vertically center everything with `align-items: center`, and add a `gap` between the individual links inside `.nav-links`.

Q6. Flexbox: card meta row

Turn `.card-meta` (inside each project card) into a flex container so the `.badge` and the "Read more" link sit on opposite ends of the row, vertically aligned. Use `justify-content` and `align-items` — no floats, no absolute positioning.

Q7. Grid: page skeleton

Turn `.page` into a CSS Grid container and lay out its five direct children using named grid areas: the header, the navbar, and the footer should each span the full width, with the sidebar and the main content sitting side-by-side in the middle row. Give the sidebar a fixed-width column and let the main content take the remaining space (`1fr`).

Q8. Grid: card gallery

Turn `.cards` into a CSS Grid using `grid-template-columns: repeat(auto-fit, minmax(220px, 1fr))` and a `gap`, so the project cards automatically wrap into as many columns as fit the screen, with no media query needed for this part.

Q9. Breakpoint 1 — activate the page grid

Wrap the grid rules you wrote in Q7 inside a `@media (min-width: ...)` query (pick a sensible breakpoint, e.g. `768px`). Below that width, `.page` should fall back to its natural stacked, single-column order (header, navbar, sidebar, main, footer, one after another) — this is the mobile-first approach from lecture: the base styles are the simple mobile layout, and the media query adds the more complex desktop layout on top.

Q10. Breakpoint 2 + fluid type

Add a **second** `@media (min-width: ...)` breakpoint, larger than the one in Q9 (e.g. `1024px`), that makes one further adjustment at wide screens — for example, widening the sidebar column or increasing the gap/spacing. Separately, make the `<h1>` in the header use `clamp()` for its `font-size` (a minimum, a preferred `vw`-based value, and a maximum) so it scales fluidly with the viewport without needing its own breakpoint.
